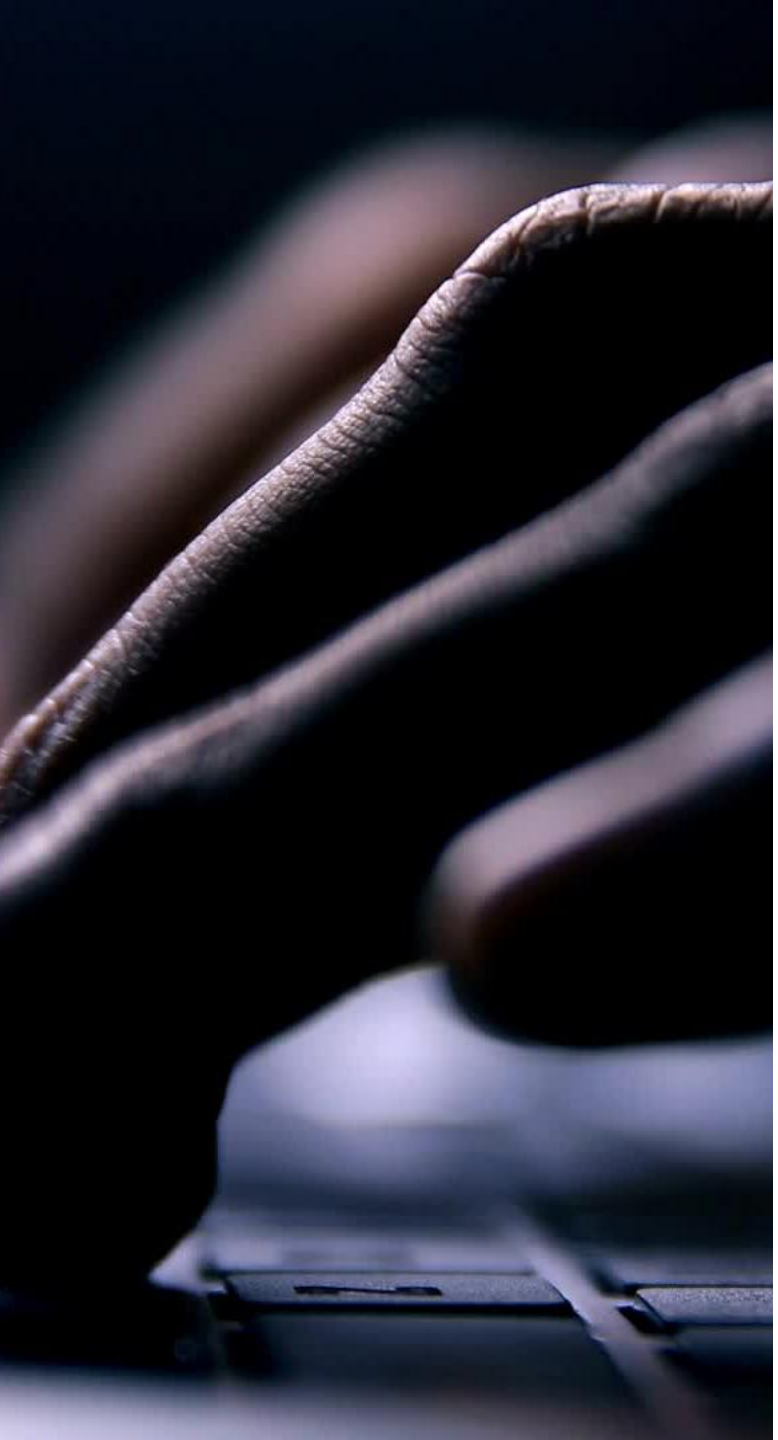


Fatec
Diadema
Luigi Papaiz

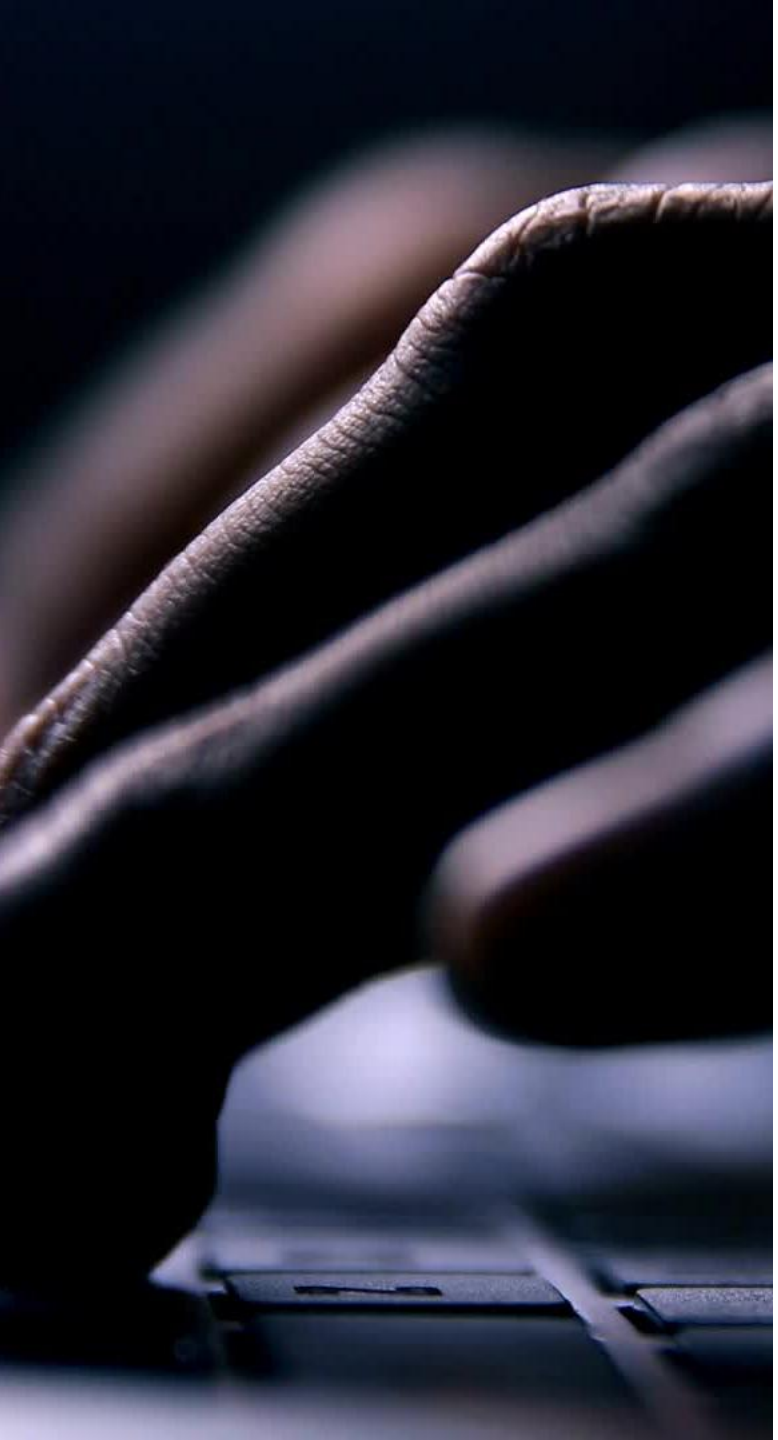
Programação para Dispositivos Móveis I

Prof. Rafael Miranda
rafael.miranda@cps.sp.gov.br

Na última aula

Fatec
Diadema
Luigi Papaiz

- Introdução ao Android Studio



Fatec
Diadema
Luigi Papaiz

Android Studio - Entendendo a estrutura

O que é uma Activity?

Fatec
Diadema
Luigi Papaiz

O sistema operacional precisa controlar:

- quando o aplicativo é aberto;
- quando uma tela aparece;
- quando o usuário sai dela;
- quando ela volta;
- quando o sistema precisa recriá-la.

Para isso, o Android trabalha com **componentes de aplicação**.

Activity é um componente Android que representa uma unidade de interação do usuário com o aplicativo.

Onde o Android encontra nossa Activity?


```
class MainActivity : ComponentActivity() {
```

MainActivity é a classe que representa nossa Activity principal.

```
import androidx.activity.ComponentActivity
```

ComponentActivity é uma classe fornecida pelo AndroidX que **herda de Activity** e acrescenta recursos/integrações da arquitetura AndroidX.

onCreate()


```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)

    setContent {
        ...
    }
}
```

- onCreate() é chamado pelo Android quando uma Activity é criada. É nesse método que colocamos o código necessário para inicializar nossa Activity.
- override indica que estamos substituindo a implementação herdada por uma implementação específica da nossa classe

super.onCreate()

Nossa MainActivity está criando seu próprio onCreate():

```
override fun onCreate(savedInstanceState: Bundle?) {

    super.onCreate(savedInstanceState)

    setContent {
        // nossa interface
    }
}
```

Mas não queremos descartar o que a classe pai já faz.

super.onCreate() executa o onCreate() da classe pai antes de continuarmos com nosso código.

savedInstanceState

Fatec
Diadema
Luigi Papaiz

savedInstanceState: Bundle?

O Android pode precisar recriar uma Activity. Esse parâmetro permite receber informações de estado que tenham sido salvas anteriormente.

- Bundle → tipo do objeto que pode conter essas informações;
- savedInstanceState → nome da variável que recebe esse objeto;
- ? → indica que pode não existir um objeto, ou seja, pode ser null.

setContent


```
setContent {

    ...

}
```

setContent define o conteúdo visual da Activity utilizando Jetpack Compose.

Tudo que colocamos dentro dele passa a fazer parte da interface que será apresentada naquela Activity.

O que é Jetpack Compose?

Jetpack Compose é o toolkit que estamos utilizando para construir a interface do aplicativo.

Em vez de criar uma tela usando arquivos XML, podemos construir a interface utilizando código Kotlin.

Por exemplo:

```
Text("Lista de Compras")
```

O que é o Theme?


```
ListaComprasTheme {

    ...
}
```

O template do Android Studio criou o Theme para centralizar características visuais do aplicativo.

Pode controlar:

- cores;
- tipografia;
- aparência dos componentes;
- tema claro/escuro.

Scaffold

Nosso código:

```
Scaffold(
    modifier = Modifier.fillMaxSize()
) { innerPadding ->

    ...

}
```

É um componente do Compose utilizado para estruturar uma tela.

Ele facilita a organização de elementos como o conteúdo principal, barra superior e inferior, botão de ação, além de outros elementos estruturais.

Modifier

Modifier permite configurar características de um componente.

Por exemplo:

```
modifier = Modifier.fillMaxSize()
```

Significa:

- Utilize o espaço disponível.

Outro exemplo que já aparece no nosso código:

```
Modifier.padding(...)
```

- Aplica espaçamento.

innerPadding

Fatec
Diadema
Luigi Papaiz

```
Scaffold { innerPadding ->

    Column(
        modifier = Modifier.padding(innerPadding)
    ) {
        ...
    }
}
```

O Scaffold fornece innerPadding para informar ao conteúdo o espaço que deve ser respeitado dentro da estrutura da tela.

Column

Column organiza os componentes **verticalmente**.

```
Column {
    Text(...)
    TextField(...)
    Button(...)
}
```

Exibir informações e entrada de dados

Fatec
Diadema
Luigi Papaiz

Text é um componente utilizado para exibir texto.

```
Text("Lista de Compras")
```

Textfield permite a entrada de dados.

```
TextField(
    value = nome,
    onValueChange = {
        nome = it
    }
)
```

onValueChange

Fatec
Diadema
Luigi Papaiz

```
onValueChange = {
    nome = it
}
```

Quando o usuário altera o texto, o TextField envia o novo valor para o código.

Esse novo valor pode ser acessado através de it.

Estado

Fatec
Diadema
Luigi Papaiz

```
var nome by remember {
    mutableStateOf("")
}
```

mutableStateOf - cria um valor que pode ser alterado e que pode ser observado pelo Compose.

remember - remember faz o Compose manter aquele valor enquanto aquele trecho da interface estiver sendo utilizado.

by - permite trabalhar diretamente com o valor do estado.

by

Sem by:

```
val nome = remember {
    mutableStateOf("")
}

nome.value
```

Com by:

```
var nome by remember {
    mutableStateOf("")
}

nome
```

Button


```
Button(
    onClick = {
        ...
    }
) {
    Text("Cadastrar")
}
```

O Button representa o componente visual.

O onClick define a ação.

O que acontece ao clicar?

No nosso aplicativo:

```
onClick = {

    val produto = Produto(
        nome = nome,
        preco = preco.toDouble(),
        quantidade = quantidade.toInt()
    )

    produtos.add(produto)

    total += produto.calcularTotal()
}
```

Nossa classe


```
class Produto(
    val nome: String,
    val preco: Double,
    val quantidade: Int
)
```

Onde ficam vários produtos?

Fatec
Diadema
Luigi Papaiz

Um produto não é suficiente. Precisamos armazenar vários.

```
val produtos = remember {
    mutableStateListOf<Produto>()
}
```

É uma lista que, além de ser mutável, é observável pelo Compose.

Como mostramos os produtos?

Fatec
Diadema
Luigi Papaiz

```
for (produto in produtos) {
    Text(
        "${produto.nome} - " +
        "${produto.quantidade} x R$ ${produto.preco}"
    )
}
```

Aqui estamos usando novamente o for e uma coleção Kotlin. O que muda é que agora essa coleção está alimentando uma interface.

Atividade - Controle de Viagens

Fatec Diadema Luigi Papaiz

Crie um aplicativo para controle de viagens, registrando os dados de cada viagem.

O aplicativo deve:

- Cadastrar data, km inicial/final, litros, combustível, pedágio e valor
- Listar as viagens realizadas
- Calcular distância percorrida
- Calcular custo de combustível + pedágios
- Mostrar total gasto, total de km e média km/l
- Limpar os campos após o cadastro